P2909R4
Fix formatting of code units as integers (Dude, where’s my char?)

Published Proposal, 2023-11-07

Author:
Victor Zverovich

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

"In character, in manner, in style, in all things, the supreme excellence is simplicity." — Henry Wadsworth Longfellow

§ 1. Introduction

The C++20 formatting facility (`std::format`) allows formatting of `char` as an integer via format specifiers such as `d` and `x`. Unfortunately [P0645] that introduced the facility didn’t take into account that signedness of `char` is implementation-defined and specified this formatting in terms of `to_chars` with the value implicitly converted (promoted) to `int`. This had some undesirable effects discovered after getting usage experience and resolved in the {fmt} library ([FMT]). This paper proposes applying a similar fix to `std::format`.

First, `std::format` normally produces consistent output across platforms for the same integral types and the same IEEE 754 floating point types. Formatting `char` as an integer breaks this nice property making the output implementation-defined even if the `char` size is effectively the same.

Second, `char` is used as a code unit type in `std::format` and other text processing facilities. In these use cases one normally needs to either output `char` as (a part of) text which is the default or as a bit pattern. Having it sometimes be output as a signed integer is surprising to users. It is particularly surprising when formatted in a non-decimal base. For example, assuming UTF-8 literal encoding:

```
for (char c : std::string("🤷")) {
  std::print("\\x{:02x}", c);
}
```

will print either

```
\xf0\x9f\xa4\xb7
```

or

```
\x-10\x-61\x-5c\x-49
```

depending on a platform. Since it is implementation-defined, the user may not even be aware of this issue which can then manifest itself when the code is compiled and run on a different platform or with different compiler flags.

This particular case can be fixed by adding a cast to `unsigned char` but it may not be as easy to do when formatting ranges compared to using format specifiers.

§ 2. Changes from R3

- Replaced `__cpp_lib_format` with a more specific `__cpp_lib_format_uchar` per LWG feedback.
- Replaced "the corresponding unsigned type" to "the unsigned version of the underlying type" per LWG feedback to better accommodate `wchar_t`.

§ 3. Changes from R2

- Added LEWG poll results.

§ 4. Changes from R1

- Added instructions to bump the `__cpp_lib_format` feature test macro per LEWG feedback.
- Added a missing cast for the case of formatting `char` as `wchar_t` per LEWG feedback.

§ 5. Changes from R0

- Changed the title from "Dude, where’s my char?" to "Fix formatting of code units as integers" per SG16 feedback.
- Added all affected format specifiers to the before/after table per SG16 feedback.
- Clarified how this compares with `printf` format specifiers.
- Added SG16 poll results for R0.
- Fixed handling of the case of formatting `char` as `wchar_t` per SG16 feedback.

§ 6. Polls

LEWG poll results for R1:

POLL: Forward P2909R1 to LWG for C++26 (and as a defect) (to be confirmed by Electronic Polling)

```
SF  F  N  A SA
 5 11  1  0  0
```

Outcome: Strong consensus in favour

POLL: For a feature test Macro we prefer a new Macro (over bumping "__cpp_lib_format")

```
SF  F  N  A SA
 0  3  4  3  2
```

Outcome: No consensus

SG16 poll results for R0:

Poll 1: Modify P2909R0 "Dude, where's my char‽" to maintain semi-consistency with printf such that the `b`, `B`, `o`, `x`, and `X` conversions convert all integer types as unsigned.

```
SF  F  N  A SA
 1  2  0  2  2
```

Outcome: No consensus for change

Poll 2: Modify P2909R0 "Dude, where's my char‽" to remove the change to handling of the `d` specifier.

```
SF  F  N  A SA
 2  1  2  1  1
```

Outcome: No consensus for change

Poll 3: Forward P2909R0 "Dude, where's my char‽", amended with a descriptive title, an expanded before/after table, and fixed CharT wording, to LEWG with the recommendation to adopt it as a Defect Report.

```
SF  F  N  A SA
 2  2  2  1  0
```

Outcome: Weak consensus - LEWG may want to look at this closely

§ 7. Proposal

This paper proposes making code unit types formatted as unsigned integers instead of implementation-defined.

<table>
<tr><th>Code</th><th>Before</th><th>After</th></tr>
<tr><td><pre>// Assuming UTF-8 as a literal encoding.
for (char c : std::string("🤷")) {
  std::print("\\x{:02x}", c);
}</pre></td><td><pre>\xf0\x9f\xa4\xb7</pre>or<pre>\x-10\x-61\x-5c\x-49</pre>(implementation-defined)</td><td><pre>\xf0\x9f\xa4\xb7</pre></td></tr>
<tr><td><pre>std::print(
  "{0:b} {0:B} {0:d} {0:o} {0:x} {0:X}",</pre></td><td><pre>11110000 11110000 240 360 f0 F0</pre></td><td><pre>11110000 11110000 240</pre></td></tr>
</table>

`'\xf0');`	or `-10000 -10000 -16 -20 -10 -10` (implementation-defined)

This somewhat improves consistency with `x` and `o` (but not `d`) `printf` specifiers which always treat arguments as unsigned. For example:

```
printf("%x", '\x80');
```

prints

```
ffffff80
```

regardless of whether `char` is signed or unsigned.

This is not a goal though but a side effect of picking a consistent platform-independent representation for code unit types. Unlike `printf`, `std::format` doesn't need to convey signedness or other type information in format specifiers. The latter is an artefact of varargs limitations.

The current paper updates the `__cpp_lib_format` feature test macro instead of introducing a new one since the amount of work to check the macro and perform different action based on it is comparable to switching to the type with signdness that doesn't depend on the implementation (`signed char` or `unsigned char`).

§ 8. Wording

Add a feature-testing macro `__cpp_lib_format_uchar` with the value set to the date of adoption in [version.syn].

Change in [tab:format.type.char]:

Table 69: Meaning of type options for `charT` [tab:format.type.char]

Type	Meaning
none, `c`	Copies the character to the output.
`b`, `B`, `d`, `o`, `x`, `X`	As specified in Table 68 with `value` converted to the unsigned version of the underlying type .
`?`	Copies the escaped character ([format.string.escaped]) to the output.

Change in [format.arg]:

```
template<class T> explicit basic_format_arg(T& v) noexcept;
```

...

Effects: Let `TD` be `remove_const_t<T>`.

- If `TD` is `bool` or `char_type`, initializes `value` with `v`;
- otherwise, if `TD` is `char` and `char_type` is `wchar_t`, initializes `value` with `static_cast<wchar_t>(`~~`v`~~`static_cast<unsigned char>(v))`;

...

§ 9. Impact on existing code

This is a breaking change but it only affects the output of negative/large code units when output via opt-in format specifiers. There were no issues reported when the change was shipped in {fmt} and the number of uses of `std::format` is orders of magnitude smaller at the moment.

§ 10. Implementation

The proposed change has been implemented in the {fmt} library ([FMT]).

§ References

§ Informative References

[FMT]
Victor Zverovich; et al. *The fmt library*. URL: https://github.com/fmtlib/fmt

[P0645]
Victor Zverovich. *Text Formatting*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0645r10.html